HMI Manager - Documentação Completa do Projeto

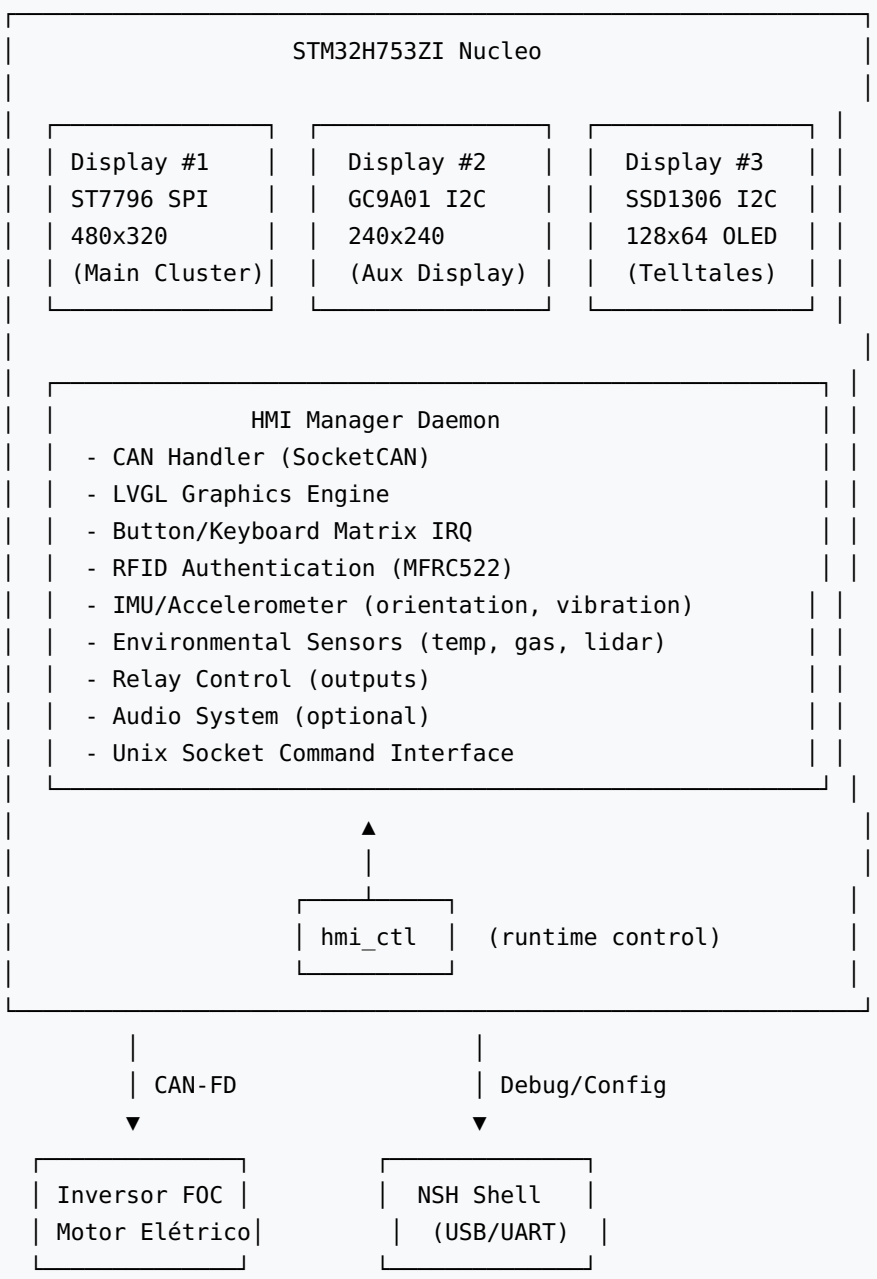
> **Projeto:** Sistema HMI para Caminhão Elétrico
> **Hardware:** STM32H753ZI Nucleo (Cortex-M7 @ 480MHz)
> **RTOS:** Apache NuttX
> **Aplicação:** Tração elétrica com inversor trifásico FOC
> **Status:** Em desenvolvimento ativo

1. Visão Geral do Sistema

1.1 Objetivo

Desenvolver um sistema HMI profissional para cluster automotivo de caminhão elétrico, com múltiplos displays, controle via botões/teclado, comunicação CAN com inversor de tração, telemetria, diagnóstico e sistema de autenticação por RFID.

1.2 Arquitetura do Sistema



2. Funcionalidades Implementadas

> **Status Atual: Sistema Base Funcional**
>
> ✓ **Display Principal (ST7796):** Cluster LVGL com velocidade, tensão, corrente, direção

- ✓ **Comunicação CAN:** RX/TX com inversor, parsing DBC automático
- ✓ **CAN Trace:** Debug profissional com filtro por ID, timestamp, syslog
- ✓ **Controle de Botões:** 11 botões com IRQ, mapeamento configurável
- ✓ **Arquitetura Daemon:** hmi_manager (background) + hmi_ctl (cliente)
- ✓ **Unix Domain Sockets:** IPC eficiente para comandos runtime
- ✓ **Framebuffer Driver:** ST7796 320x480 RGB565 via SPI
- ✓ **LVGL Integration:** Graphics library com widgets customizados
- ✓ **Modular Codebase:** Estrutura profissional (can/, io_handler/, uiux/, cmd/)
- ✓ **NuttX Coding Style:** Compliance total, 78 colunas, headers Apache

2.1 Módulos do Sistema

Módulo	Arquivo	Função	Status
Daemon Principal	`app_hmi_manager.c`	Loop principal, coordenação de todos os subsistemas	✅ Funcional
Cliente de Controle	`hmi_ctl.c`	Envia comandos ao daemon via socket	✅ Funcional
CAN Handler	`can/can_handler.c`	Wrapper SocketCAN, send/recv non-blocking	✅ Funcional
CAN Protocol	`can/car_can.c/h`	DBC gerado (cantools), 10 mensagens, 30 sinais	✅ Funcional
CAN Trace	`can/can_trace.c`	Logging profissional, filtro por ID, syslog	✅ Funcional
Button IRQ	`io_handler/button_irq.c`	Tratamento de IRQ, debouncing, 11 botões	✅ Funcional
Framebuffer Handler	`uiux/fb_handler.c`	Inicialização LVGL, display ST7796	✅ Funcional
Widgets LVGL	`uiux/widgets.c`	Speed, voltage, current, direction labels	✅ Funcional
Command Parser	`cmd/commands.c`	Parse argc/argv, routing para handlers	✅ Funcional
Logger	`logs/logger.c`	Sistema de log com níveis (ERROR, WARN, INFO, DEBUG)	🔄 Stub
RFID Handler	`modules/rfid.c`	MFRC522 reader, autenticação de tags	🔄 Stub
Accelerometer	`modules/accel.c`	IMU, detecção de orientação, vibração	🔄 Stub
Aux Display	`uiux/aux_disp.c`	Display secundário (GC9A01)	🔄 Stub
Notifications	`uiux/notifications.c`	Telltales, alertas, OLED SSD1306	🔄 Stub
Config Manager	`config/config_manager.c`	Persistência de configuração (tags RFID, calibração)	🔄 Stub

3. API do HMI Manager

3.1 Inicialização e Controle

Iniciar o Daemon

```
nsh> hmi_manager &
```

O que acontece:

1. Inicializa display ST7796 (framebuffer + LVGL)
2. Abre interface CAN (can0, SocketCAN)
3. Inicializa sistema de trace CAN (desabilitado)
4. Configura IRQ dos botões (/dev/buttons)
5. Cria Unix socket (/tmp/hmi_cmd)
6. Cria widgets LVGL (speed, voltage, current, direction)
7. Entra no loop principal (select + LVGL + IRQ)

Verificar Status

```
nsh> ps PID PRI POLICY TYPE NPX STATE USED STACK COMMAND 4 100 RR Task --- Running 12456/16384 hmi_manager
```

Parar o Daemon

```
nsh> kill 4
```

3.2 API de Comandos (hmi_ctl)

CAN Trace

Comando	Descrição	Exemplo
`hmi_ctl can --trace show`	Habilita logging de todas mensagens CAN	Mostra timestamp, ID, DLC, dados
`hmi_ctl can --trace hide`	Desabilita trace CAN	Para de logar mensagens
`hmi_ctl can --trace filter 0x123`	Filtra por ID específico (hex ou decimal)	Mostra apenas ID 0x123
`hmi_ctl can --trace filter 0`	Remove filtro	Volta a mostrar todos os IDs

System Logging

Comando	Descrição
`hmi_ctl log --show`	Habilita logs do sistema
`hmi_ctl log --hide`	Desabilita logs do sistema

Help

```
nsh> hmi_ctl --help Usage: hmi_ctl [args...] Commands: can --trace show Enable CAN trace can --trace hide Disable CAN trace can --trace filter Filter by CAN ID (0 = all) log --show Enable system log log --hide Disable system log --help Show this help
```

3.3 API Interna (Programação)

CAN Handler

```
#include "can/can_handler.h" /* Inicializar */ int can_handler_init(const char *ifname); /* Enviar frame */ int can_handler_send(uint32_t id, bool extended, const uint8_t *data, uint8_t dlc); /* Receber frame (non-blocking) */ int can_handler_read(struct can_frame *frame); /* Obter file descriptor (para select) */ int can_handler_get_fd(void); /* Fechar */ void can_handler_close(void);
```

CAN Trace

```
#include "can/can_trace.h" /* Inicializar sistema de trace */ void can_trace_init(void); /* Habilitar/desabilitar */ void can_trace_enable(bool enable); /* Verificar se está habilitado */ bool can_trace_is_enabled(void); /* Logar frame RX */ void can_trace_rx(const struct can_frame *frame); /* Logar frame TX */ void can_trace_tx(const struct can_frame *frame); /* Definir filtro por ID */ void can_trace_set_filter(uint32_t id_mask);
```

DBC Protocol

```
#include "can/car_can.h" /* Exemplo: Enviar comando de marcha */ struct car_can_driver_commands_t cmd; uint8_t data[8]; int dlc; car_can_driver_commands_init(&cmd); cmd.demanded_drive_direction = 0; /* Forward */ cmd.demanded_drive_state = 1; /* Enable */ dlc = car_can_driver_commands_pack(data, &cmd, sizeof(data)); can_handler_send(CAR_CAN_DRIVER_COMMANDS_FRAME_ID, CAR_CAN_DRIVER_COMMANDS_IS_EXTENDED, data, dlc); /* Exemplo: Receber tensão da bateria */ struct can_frame frame; can_handler_read(&frame); if ((frame.can_id & CAN_EFF_MASK) == CAR_CAN_SYSTEM_VOLTAGES_FRAME_ID) { struct car_can_system_voltages_t msg; car_can_system_voltages_unpack(&msg, frame.data, frame.can_dlc); double voltage = car_can_system_voltages_meas_batt_voltage_decode( msg.meas_batt_voltage); printf("Battery: %.1f V\n", voltage); }
```

Widgets LVGL

```
#include "uiux/widgets.h" /* Inicializar todos os widgets */ int widgets_init(void); /* Atualizar velocidade (km/h) */ void widgets_set_speed(int kmh); /* Atualizar tensão (decivolts, ex: 485 = 48.5V) */ void widgets_set_voltage(int dv); /* Atualizar corrente (deciamps, ex: 155 = 15.5A) */ void widgets_set_current(int da); /* Atualizar direção */ void widgets_set_direction(int dir); /* dir: WIDGET_DIR_NEUTRAL, WIDGET_DIR_FORWARD, WIDGET_DIR_REVERSE */
```

Button IRQ

```
#include "io_handler/button_irq.h" /* Inicializar */ int button_irq_init(const char *devpath); /* Verificar botão pressionado */ int button_irq_get_pressed(void); /* Retorna: 0-10 (botão) ou BUTTON_INVALID */ /* Limpar flag */ void button_irq_clear(void); /* Fechar */ void button_irq_close(void);
```

3.4 Exemplos de Uso Completos

Exemplo 1: Monitorar CAN Bus

```
/* Terminal NuttX */ nsh> hmi_manager & nsh> hmi_ctl can --trace show /* Terminal Linux (conectado no CAN) */ $ cansend can0 0C06FE00#000000000000F212 /* Saída no NuttX */ timestamp | interface | id | dlc | data ---------------------------------------------- 1230 | can0 | 0C06FE00 | 8 | 00 00 00 00 00 00 F2 12 /* Display mostra: "48.5 V" */
```

Exemplo 2: Debug de Mensagem Específica

```
nsh> hmi_ctl can --trace filter 0x0C09FEFE nsh> hmi_ctl can --trace show /* Agora só mostra InverterDemanders (velocidade/pedal) */ /* Linux */ $ cansend can0 0C09FEFE#0100000000002003 /* NuttX mostra apenas essa mensagem */ 2340 | can0 | 0C09FEFE | 7 | 01 00 00 00 00 20 03
```

Exemplo 3: Controlar Veículo via Botões

```
/* Usuário pressiona botão físico 1 */ /* IRQ dispara */ /* Daemon detecta: button_irq_get_pressed() = 1 */ /* Envia CAN: DriverCommands com direction=FORWARD, state=ENABLE */ /* No Linux, candump mostra: */ $ candump can0 can0 0C03FE32 [1] 02 /* Inversor recebe comando e liga motor em marcha à frente */
```

Exemplo 4: Adicionar Novo Sinal no Display

```
/* 1. Abrir car_can.h */ /* 2. Encontrar mensagem: INVERTER_TEMP */ /* 3. Ver struct: inverter_temp_u, inverter_temp_v, inverter_temp_w */ /* 4. Adicionar em app_hmi_manager.c, função process_can_frame(): */ else if (id == CAR_CAN_INVERTER_TEMP_FRAME_ID) { struct car_can_inverter_temp_t msg; ret = car_can_inverter_temp_unpack(&msg, frame.data, frame.can_dlc); if (ret == 0) { /* Decode temperatura fase U */ double temp = car_can_inverter_temp_inverter_temp_u_decode( msg.inverter_temp_u); /* Atualizar widget (criar novo ou reutilizar existente) */ widgets_set_temperature((int)temp); } } /* 5. Criar função widgets_set_temperature() em widgets.c */ /* 6. Adicionar label LVGL para temperatura */ /* 7. Recompilar */
```

Exemplo 5: Configurar Novo Botão

```
/* Editar app_hmi_manager.c, adicionar na tabela g_button_map[] */ /* BTN5: Reset de falhas */ { .msg_id = CAR_CAN_DEBUG_DEMANDS_FRAME_ID, .extended = CAR_CAN_DEBUG_DEMANDS_IS_EXTENDED, .data = {0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00}, .dlc = 1 }, /* Agora botão 5 envia reset quando pressionado */
```

4. Roadmap de Desenvolvimento

4.1 Features Planejadas

Display Multi-Tela

- ⚡ **Display Principal (ST7796 480x320):** Cluster completo com velocímetro, hodômetro, indicadores principais
- ⚡ **Display Auxiliar (GC9A01 240x240):** Informações secundárias, navegação, câmera ré
- ⚡ **Display Telltales (SSD1306 128x64 OLED):** Warnings, alertas críticos, ícones monocromáticos
- ⚡ **Dual Framebuffer:** Double buffering para animações suaves, reduz flickering
- ⚡ **Display On/Off via NSH:** Comando `hmi_ctl display --off` para economizar energia

Drivers a Desenvolver

- ⚡ **GC9A01 I2C Framebuffer Driver:** TFT circular 1.28" 240x240, ideal para display auxiliar
- ⚡ **Relay Control Driver:** Controle de relés para luzes, ventiladores, buzzer
- ⚡ **Keyboard Matrix Driver:** Substituir GPIO individual por matriz de teclado (economiza pinos)
- ⚡ **ST7796 Touch Driver:** Adicionar suporte a touchscreen (XPT2046 ou FT6236)

- ⚡ **Digital Potentiometer Driver:** Controle de volume, brilho via I2C/SPI

Módulos de Sensores

- ⚡ **Acelerômetro/IMU:** Detectar orientação do veículo, vibração, acidentes
- ⚡ **Sensor de Gás (MQ-x):** Detecção de vazamento de combustível, monóxido de carbono
- ⚡ **LiDAR VL53L5CX:** Sensor de distância multi-zona para assistência de estacionamento
- ⚡ **DS18B20 1-Wire:** Múltiplos sensores de temperatura (motor, bateria, cabine)

Sistema de Autenticação

- ⚡ **RFID Login (MFRC522):** Tag RFID/NFC para ligar sistema, múltiplos usuários
- ⚡ **Cadastro de Tags via Menuconfig:** Configuração de tags autorizadas no Kconfig
- ⚡ **Timeout de Inatividade:** Desliga display/sistema após X minutos sem tag
- ⚡ **Log de Acessos:** Registra quando cada tag foi usada (auditoria)

Sistema de Áudio (Opcional)

- ⚡ **I2S Audio Output:** Saída para amplificador classe D
- ⚡ **Buzzer/Beep:** Alertas sonoros (warning, critical, info)
- ⚡ **Text-to-Speech:** Avisos falados via síntese de voz
- ⚡ **Controle de Volume:** Via potenciômetro digital ou touch

4.2 Organização dos Displays

Display	Tipo	Resolução	Interface	Conteúdo
Display #1 (Principal)	ST7796 TFT	480x320	SPI	• Velocímetro grande (0-120 km/h) • Hodômetro (km total, trip A/B) • Tensão da bateria (gráfico barra) • Corrente do motor (instantânea) • Estado de carga (SOC %) • Direção/marcha (PRND) • Temperatura (motor, inversor, bateria) • Hora/data
Display #2 (Auxiliar)	GC9A01 TFT Circular	240x240	I2C	• Câmera de ré (ao engatar marcha ré) • Consumo energético (kWh/km) • Autonomia estimada (km restantes) • Gráfico de potência (últimos 60s) • Info de navegação (opcional) • Status de sensores (lidar, gás, etc)
Display #3 (Telltales)	SSD1306 OLED	128x64	I2C	• ⚠ Ícones de alerta (10-15 ícones) • Bateria baixa • Temperatura alta • Falha no inversor • Pressão dos pneus • Porta aberta • Freio de mão

				• Check engine • Luzes (farol alto, neblina) • Setas (esquerda/direita)

Por que 3 displays?
- **Separação de Responsabilidades:** Principal (info driving), Auxiliar (telemetria avançada), Telltales (crítico/segurança)
- **Redundância:** Se display principal falhar, telltales ainda mostra alertas críticos
- **Eficiência:** OLED monocromático consome muito menos energia, sempre ligado
- **Profissionalismo:** Layout similar a veículos modernos (Tesla, Rivian, etc)

4.3 Desafios Técnicos

Desafio	Complexidade	Solução Proposta
GC9A01 I2C Framebuffer Driver	Alta	• Estudar datasheet do GC9A01 (comandos de inicialização) • Implementar driver NuttX estilo /drivers/lcd/ • Criar interface framebuffer (/dev/fb1) • Integrar com LVGL (segundo display) • Otimizar taxa de atualização via DMA
Keyboard Matrix vs GPIO	Média	• Avaliar número de botões necessários (12-16?) • Matriz 4x4 = 8 GPIOs vs 16 GPIOs individuais • Implementar driver de matriz com debouncing • IRQ em linhas de entrada • Trade-off: Economiza GPIOs mas adiciona latência (~1ms)
Pinout Disponível STM32H753ZI	Alta	• Criar planilha com todos os periféricos usados • Listar GPIOs: SPI1 (ST7796), I2C2 (GC9A01, SSD1306, MFRC522), CAN, UART, ADC, etc • Verificar conflitos (ex: SPI1_MOSI usado por 2 devices) • Priorizar periféricos essenciais • Considerar multiplexação via I2C expander (PCF8574)
Touchscreen ST7796	Média	• Identificar chip do touch (XPT2046 ou FT6236) • XPT2046: Driver SPI já existe no NuttX (/drivers/input/) • FT6236: Driver I2C, pode precisar adaptação • Calibração: Criar rotina de 4 pontos • Integrar com LVGL (lv_indev_drv_t)
Relay Control	Baixa	• Driver simples: GPIO output + lógica invertida (se necessário) • Criar /dev/relay ou interface ioctl • Proteção: PWM para soft-start (reduz inrush current) • Feedback: Ler corrente para detectar falha no relé
RFID Authentication	Média	• Driver MFRC522 já existe (stm32_mfrc522.c na board) • Implementar lógica: ler tag → verificar whitelist → autorizar/negar • Whitelist em Kconfig ou arquivo de config persistente • Timeout: Após X segundos sem tag, desabilita sistema • Visual feedback: LED verde/vermelho ou mensagem no display
Múltiplos Sensores de		• Driver DS18B20 já implementado na board • 1-Wire permite múltiplos sensores no mesmo pino

Temperatura (1-Wire)	▧ Baixa	• Buscar todos os dispositivos (ROM search) • Ler temperatura de cada um sequencialmente • Mostrar no display: Motor 85°C, Bateria 42°C, Cabine 23°C
Acelerômetro (Uso?)	▩ Média	**Aplicações:** • Detecção de inclinação (subida/descida, compensar velocímetro) • Detecção de acidente/colisão (disparar alerta, blackbox) • Análise de condução (acelerações bruscas, frenagens) • Monitoramento de vibração (manutenção preditiva) **Implementação:** • IMU I2C (MPU6050, ADXL345, etc) • Leitura periódica (10-100 Hz) • Filtro passa-baixa para remover ruído • Threshold para detectar eventos
Sistema de Áudio	◍ Alta	**Desafio:** STM32H753ZI tem I2S mas requer DAC/codec externo **Alternativas:** • PWM Audio: Qualidade baixa mas simples (beeps, alertas) • I2S + MAX98357A: Amplificador classe D, boa qualidade • Buzzer piezo: Simples, apenas para alertas **Prioridade:** Baixa (não essencial para MVP)
Performance LVGL com 3 Displays	◍ Alta	• LVGL consome bastante CPU (especialmente com animações) • STM32H753 @ 480 MHz deve dar conta, mas precisa otimizar: • - Usar framebuffer em DTCM/SRAM (acesso rápido) • - Atualizar displays em prioridades diferentes • - Display principal: 30 FPS, Auxiliar: 15 FPS, OLED: 5 FPS • - Usar double buffering apenas no display principal • - Considerar usar DMA2D para cópia de framebuffer
Potenciômetros Digitais	▧ Baixa	**Uso:** • Controle de brilho dos displays • Controle de volume do áudio **Chips comuns:** MCP4131 (SPI), AD5242 (I2C) **Implementação:** Driver simples, escreve valor 0-255 via I2C/SPI

4.4 Priorização de Features (MoSCoW)

Prioridade	Feature	Justificativa
Must Have	• Display principal funcional (ST7796) • CAN RX/TX com inversor • Controle de direção (botões/teclado) • Telltales críticos (OLED) • Sistema de trace/debug	Essencial para MVP funcional e seguro
Should Have	• Display auxiliar (GC9A01) • RFID authentication • Sensores de temperatura (1-Wire) • Relay control (luzes, buzzer) • Touchscreen	Aumenta usabilidade e profissionalismo
Could Have	• Acelerômetro/IMU • Sensor de gás • LiDAR VL53L5CX	Nice to have, adiciona features avançadas

	• Potenciômetros digitais • Keyboard matrix	
Won't Have (v1.0)	• Sistema de áudio • Câmera de ré • Navegação GPS • Conectividade (WiFi, Bluetooth)	Complexidade alta, pode ser v2.0

5. Recomendações de Implementação

5.1 Roadmap Incremental

Fase 1: Consolidar Base (2-3 semanas)

1. Remover todos os stubs, implementar módulos básicos
2. Mapear todos os sinais CAN corretos do DBC
3. Criar widgets LVGL profissionais (gauges, gráficos)
4. Testar exaustivamente comunicação CAN
5. Documentar API completa

Fase 2: Display Auxiliar + Telltales (2-3 semanas)

1. Desenvolver driver GC9A01 I2C framebuffer
2. Integrar segundo display com LVGL
3. Implementar display OLED telltales (ícones)
4. Criar sistema de notificações (prioridades)
5. Testar multi-display simultâneo

Fase 3: RFID + Sensores (1-2 semanas)

1. Implementar autenticação RFID
2. Integrar sensores de temperatura (DS18B20)
3. Adicionar relay control para outputs
4. Criar sistema de configuração persistente

Fase 4: Features Avançadas (2-4 semanas)

1. Implementar keyboard matrix (se necessário)
2. Adicionar touchscreen
3. Integrar acelerômetro (se uso definido)
4. Otimizar performance (DMA, double buffer)
5. Polimento final (animações, UX)

5.2 Arquitetura de Software Recomendada

Separação por Displays

```
uiux/
```

```
├── fb_handler.c          # Display principal (ST7796)
├── aux_display.c         # Display auxiliar (GC9A01) - NOVO
├── telltales.c           # Display OLED (SSD1306) - NOVO
├── widgets/
│   ├── main_cluster.c    # Widgets do display principal
│   ├── aux_widgets.c     # Widgets do display auxiliar
│   └── icons.c           # Ícones telltales (bitmap)
└── lvgl_multi_display.c  # Gerenciador multi-display
```

Gerenciamento de Prioridades

```
Main Loop (10ms tick):
├── [Prioridade 1] Processar CAN crítico (safety, faults)
├── [Prioridade 2] Atualizar telltales OLED (se mudança)
├── [Prioridade 3] Atualizar display principal (30 FPS)
├── [Prioridade 4] Atualizar display auxiliar (15 FPS)
├── [Prioridade 5] Processar comandos IPC
└── [Prioridade 6] Tasks background (logging, config save)
```

5.3 Pinout Recommendation

> **Antes de escalar hardware, faça mapeamento completo:**
>
> 1. Liste TODOS os periféricos necessários
> 2. Verifique datasheet do STM32H753ZI (pinout alternate functions)
> 3. Use STM32CubeMX para validar configuração
> 4. Documente pinout final em planilha Excel
> 5. Considere usar I2C expanders (PCF8574) se ficar sem GPIOs

5.4 Testes e Validação

- **Unit Tests:** Testar cada módulo isoladamente (can_handler, widgets, etc)
- **Integration Tests:** Testar comunicação entre módulos
- **Stress Tests:** CAN flood (1000 msgs/s), verificar se sistema aguenta
- **Endurance Tests:** Rodar por 24h+ sem falhas
- **Safety Tests:** Desligar CAN, verificar se telltales alertam
- **Performance Tests:** Medir CPU usage, stack usage, frame rate

6. Conclusão

> ### Sistema Atual: Base Sólida
>
> O HMI Manager já possui uma arquitetura profissional, modular e escalável. A base está pronta para adicionar features avançadas sem comprometer qualidade ou manutenibilidade.
>
> ### Próximos Passos Críticos
>
> 1. **Validar pinout:** Confirmar que STM32H753ZI tem GPIOs suficientes
> 2. **Desenvolver driver GC9A01:** Display auxiliar circular
> 3. **Implementar RFID:** Sistema de autenticação
> 4. **Otimizar LVGL:** Multi-display com performance adequada

5. **Testar exaustivamente:** Validar sistema em condições reais

Recomendação Final

Foque no MVP primeiro: Display principal + CAN + Telltales funcionando perfeitamente. Depois escale incrementalmente adicionando features uma por vez, sempre testando estabilidade antes de prosseguir.

Documentação Viva:
Este documento deve ser atualizado conforme o projeto evolui. Mantenha sempre sincronizado com o código real.